

System and Device Programming

Standard Exam

11.07.2024

Ex 1 (1.25 points)

Analyze the following code snippet in C++. Indicate all possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
template<typename Container, typename Compare>
void sc (Container& container, Compare comp) {
    container.sort(comp);
}

int main() {
    std::list<std::string> list = {
        "apple", "orange", "banana", "grape", "pineapple";
    auto dor = [](const std::string& a, const std::string& b) {
        return a > b;
    };
    sc (list, dor);
    for (const auto & str : list) {
        std::cout << str << " ";
    }
    return 0;
}
```

Choose one or more options:

1. ☐ apple banana grape orange pineapple
2. ☐ 5 6 5 6 9
3. ☐ 9 6 5 6 5
4. ☐ apple orange banana grape pineapple
5. ☒ pineapple orange grape banana apple
6. ☐ pineapple grape banana orange apple
7. ☐ 5 6 6 5 9
8. ☐ 9 5 6 6 5

Ex 2 (1.25 points)

Analyze the following code snippet in C++. Indicate all possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
int main() {
    std::map<std::string, std::set<int>> myMap;
    myMap["third"].insert(6);
    myMap["third"].insert(8);
    myMap["third"].insert(7);
    myMap["second"].insert(4);
    myMap["second"].insert(5);
    myMap["first"].insert(2);
    myMap["first"].insert(1);
    myMap["first"].insert(3);
    for (const auto& pair : myMap) {
        std::cout << pair.first << ": ";
        for (const auto& num: pair.second) {
```

```

        std::cout << num << " ";
    }
}
std::string key = "second";
if (myMap.find(key) != myMap.end()) {
    for (const auto& num : myMap[key]) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
} else {
    std::cout << "Key \"" << key << "\" not found." << std::endl;
}
return 0;
}

```

Choose one or more options:

1. ☐ 1 2 3 4 5 6 7 8 4 5
2. ☐ first: second: third:
3. ☐ first: 1 2 3 second: 4 5 third: 6 7 8
4. ☐ first: second: third: 4 5
5. ☒ first: 1 2 3 second: 4 5 third: 6 7 8 4 5
6. ☐ first: 2 1 3 second: 4 5 third: 6 8 7 4 5
7. ☐ third: 6 7 8 second: 4 5 first: 1 2 3 4 5
8. ☐ first: 2 1 3 second: 4 5 third: 6 8 7 4 5

Ex 3 (1.0 points)

Analyze the following code snippet reporting the use of a condition variable implemented in the library Pthread. Indicate which of the following statements are correct. Note that wrong answers imply a penalty in the final score.

```

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int sharedData = 0;
int ready = 0;

void* producer(void* arg) {
    pthread_mutex_lock(&mutex);
    sleep(1);
    sharedData = 42;
    ready = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void* consumer(void* arg) {
    pthread_mutex_lock(&mutex);
    while (ready == 0) {
        pthread_cond_wait(&cond, &mutex);
    }
    printf("Consumer: consumed data = %d\n", sharedData);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

```

```

int main() {
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}

```

Choose one or more options:

1. ☒ The `pthread_cond_wait` function in the consumer thread waits for the condition variable to be signaled, and, in the meantime, it releases the mutex.
2. ☐ If `pthread_cond_signal` is called before `pthread_cond_wait`, the consumer thread will immediately proceed after calling `pthread_cond_wait`.
3. ☒ If `pthread_cond_signal` is called before `pthread_cond_wait`, the signal will be lost, and the consumer thread may wait indefinitely.
4. ☒ If `pthread_mutex_unlock(&mutex)` is not called in the producer thread after `pthread_cond_signal`, the consumer thread might be blocked indefinitely.
5. ☐ If `pthread_mutex_unlock(&mutex)` is not called in the producer thread after `pthread_cond_signal`, the program would produce a compile-time error.
6. ☐ The `while (ready==0)` in the consumer thread can be substituted by `if (ready==0)`.
7. ☐ If more producers are run by the main, the condition variable synchronization does not work.
8. ☐ If more consumers are run by the main, the condition variable synchronization works only if `pthread_cond_signal` is substituted by `pthread_condition_broadcast`.
9. ☒ The shared variable `sharedData` is protected by a mutex to avoid race conditions.

Ex 4 (3.5 points)

One C Unix process named P forks a child process C. P and C processes share a chunk of memory to exchange information and use a pipe to synchronize their effort. They behave as follows:

- P starts first.
 1. It generates a random number of float values and writes them on the shared memory.
 2. Then, it sends a token on the pipe to wake up C, who waits on the pipe.
 3. Finally, it waits for a token on the same pipe, and when the token is received, it restarts from step 1.
- C starts after P.
 1. It waits for a token on the pipe.
 2. Once it has received the token, it displays the floats stored in the shared memory on standard output, and it sends a token to P to state it has done.
 3. It iterates, resparting from step 1.

Notice that the number of float values stored in the shared memory is variable (but must be stored in the shared memory); thus, it can be communicated from P to C using the pipe. Implement in C code either the parent process P or the child process C, i.e., it is not requested to implement both of them.

Solution

```

#include <iostream>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

```

```

#include <chrono>
#include <thread>

using namespace std;

#define SHM_SIZE 128
#define MAX_NUM 10

void parent (int [], int [], float *);
void child (int [], int [], float *);

int main () {
    int shmid;
    key_t key;
    float *r;
    int fd1[2], fd2[2], childPid;

    if ((key = ftok("hello.txt", 3)) == -1) {
        cerr << "Error in ftok." << endl;
        exit (1);
    }

    /* Create the segment */
    if ((shmid = shmget(key, SHM_SIZE, 0644 | IPC_CREAT)) == -1) {
        cerr << "Error in shmid." << endl;
        exit (1);
    }

    /* Attach to the segment to get a pointer to it */
    r = reinterpret_cast <float *> (shmat (shmid, NULL, 0));
    if (r == (float *) (-1)) {
        cerr << "Error in shmat." << endl;
        exit (1);
    }

    pipe(fd1);
    pipe(fd2);

    childPid = fork();

    if (childPid == 0) {
        child (fd1, fd2, r);
    } else {
        parent (fd1, fd2, r);
    }

    wait (NULL);

    return (1);
}

void parent (int fd1[2], int fd2[2], float *r) {
    unsigned short stop;

```

```

    srand(getpid());

    close (fd1[0]); //close read end
    close (fd2[1]); //close write end

    while(true) {
        // parent writes
        stop = rand()%MAX_NUM;
        cout << endl << ">>>> PARENT writes " << stop << " floats:" << endl;
        for (int i = 0; i < stop; ++i) {
            float n = static_cast<float>(rand()) / rand();
            cout << n << " ";
            r[i] = n;
        }

        cout.flush();
        write(fd1[1], &stop, sizeof(short));
        this_thread::sleep_for(chrono::seconds(1));
    }
}

void child (int fd1[2], int fd2[2], float *r) {
    unsigned short stop; //, temp;

    srand(getpid());

    close (fd1[1]);
    close (fd2[0]);

    while (true) {
        // child reads
        read (fd1[0], &stop, sizeof(stop));
        cout << endl << ">>>> CHILD receives " << stop << " floats:" << endl;
        for (int i = 0; i < stop; ++i) {
            float n = r[i];
            cout << n << " ";
        }
        cout.flush();
    }
}

```

Ex 5 (2.0 points)

Explain how the dynamic memory allocation is managed in C++ code using standard and smart pointers. More specifically, illustrates the different types of pointers and allocation strategies. Describe the two approaches' main characteristics, advantages, and disadvantages.

Solution

Dynamic memory allocation in C++ can be managed using standard pointers and smart pointers. Each approach has its own characteristics, advantages, and disadvantages. Below is a detailed explanation of both methods:

Standard Pointers:

- Standard Pointers: These are raw pointers that directly hold the memory address of the allocated memory.

- Syntax: Memory allocation is done using the new keyword, and deallocation is done using the delete keyword. Array deallocation requires delete [].

```
int* ptr = new int;    // Allocate memory
*ptr = 10;            // Use the allocated memory
delete ptr;           // Deallocate memory
```

C functions malloc and calloc can be used as well.

Advantages

- Control: Provides direct control over memory allocation and deallocation.
- Performance: Generally faster as there is no overhead associated with automatic memory management.

Disadvantages

- Memory Leaks: If delete is not called, it leads to memory leaks.
- Dangling Pointers: If a pointer is deleted and then accessed, it leads to undefined behavior.
- Manual Management: Requires explicit handling of memory, increasing the risk of errors.

Smart Pointers

- Smart Pointers: These are template classes in the C++ Standard Library that manage the lifetime of dynamically allocated objects.
- Types: The main types are std::unique_ptr, std::shared_ptr, and std::weak_ptr.

std::unique_ptr manages a single object. Ensures that only one unique_ptr can own the object at a time.

```
std::unique_ptr<int> ptr = std::make_unique<int>(10);
```

std::shared_ptr manages shared ownership of an object. Multiple shared_ptrs can own the same object, and the object is deallocated when the last shared_ptr is destroyed (a counter is used to know the number of pointers for the object).

```
std::shared_ptr<int> ptr1 = std::make_shared<int>(10); // Allocate and
initialize
std::shared_ptr<int> ptr2 = ptr1; // Shared ownership
```

std::weak_ptr provides a non-owning reference to an object managed by shared_ptr. Useful for breaking circular references.

```
std::shared_ptr<int> sharedPtr = std::make_shared<int>(10);
std::weak_ptr<int> weakPtr = sharedPtr; // Non-owning reference
```

Advantages

- Automatic Memory Management: Automatically deallocates memory when the pointer goes out of scope.
- Safety: Reduces the risk of memory leaks and dangling pointers.
- Convenience: Simplifies code by handling memory management automatically.

Disadvantages

- Overhead: Introduces additional overhead due to reference counting and other mechanisms.
- Complexity: Can be more complex to understand and use correctly, especially with shared_ptr and weak_ptr.

Ex 6 (3.5 points)

Write a program in C++ implementing a producer and consumer scheme using promises and futures. Two producers generate real values at random intervals (ranging from 1 to 5 seconds). When both values are ready, two consumers get those values, and then the first one displays the sum of those values, and the second one shows their product. Once the consumers have done so, the entire cycle **in the main function** restarts (with the

producers producing two values, etc.). The program ends when the sum or product of the two values (computed by the consumers) equals zero.

Solution

```
#include <iostream>
#include <thread>
#include <random>
#include <chrono>
#include <future>
#include <mutex>
#include <cmath>

std::mutex mtx, rng_mtx;
const double EPSILON = 0.0001;

// Producer function
void producer(std::promise<double>&& promise, std::uniform_real_distribution<>& r,
std::uniform_int_distribution<>& i, std::mt19937 & g) {
    int sleep_time;
    double value;
    // Sleep for a random time between 1 and 5 seconds
    {
        std::lock_guard<std::mutex> lock(rng_mtx);
        sleep_time = i(g);
    }
    std::this_thread::sleep_for(std::chrono::seconds(sleep_time));

    // Produce a value
    {
        std::lock_guard<std::mutex> lock(rng_mtx);
        value = r(g);
    }
    {
        std::lock_guard<std::mutex> lock(mtx);
        std::cout << "Producer produced value: " << value << std::endl;
    }
    // Set the value in the promise
    promise.set_value(value);
}

// Consumer function
void consumer(std::shared_future<double>& future1, std::shared_future<double>&
future2, std::promise<double> &res, bool isSum) {
    // Wait for the value from both futures
    double value1 = future1.get();
    double value2 = future2.get();

    if (isSum) {
        double sum = value1 + value2;
        {
            std::lock_guard<std::mutex> lock(mtx);
            std::cout << "Consumer sum: " << sum << std::endl;
        }
        res.set_value(sum);
    } else {
```

```

    double product = value1 * value2;
    {
        std::lock_guard<std::mutex> lock(mtx);
        std::cout << "Consumer product: " << product << std::endl;
    }
    res.set_value(product);
}

}

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis_r(1.0, 100.0);
    std::uniform_int_distribution<> dis_i(1, 5);

    bool end = false;

    while (!end) {
        // Create promises and futures
        std::promise<double> promise1;
        std::promise<double> promise2;
        std::shared_future<double> sf1 = promise1.get_future();
        std::shared_future<double> sf2 = promise2.get_future();
        std::promise<double> p_sum, p_prod;
        std::future<double> f_sum = p_sum.get_future();
        std::future<double> f_prod = p_prod.get_future();

        // Create producer threads
        std::thread producer1(producer, std::move(promise1), std::ref(dis_r),
std::ref(dis_i), std::ref(gen));
        std::thread producer2(producer, std::move(promise2), std::ref(dis_r),
std::ref(dis_i), std::ref(gen));

        // Create consumer threads
        std::thread consumer1(consumer, std::ref(sf1), std::ref(sf2),
std::ref(p_sum), true);
        std::thread consumer2(consumer, std::ref(sf1), std::ref(sf2),
std::ref(p_prod), false);

        // Wait for threads to finish
        producer1.join();
        producer2.join();
        consumer1.join();
        consumer2.join();

        if (std::abs(f_sum.get()) < EPSILON || std::abs(f_prod.get()) < EPSILON)
end = true;
    }

    return 0;
}

```

Ex 7 (2.5 points)

A class C includes as a private attribute a pointer to an array of integer values. Write the class, defining the content of the constructor, copy constructor, copy assignment operator, move constructor, move assignment operator, and destructor. Define a small main which defines an object of such a type and activates the various copy constructors.

Solution

```
#include <iostream>
#include <algorithm> // For std::copy

class C {
private:
    int* data;
    size_t size;

public:
    // Constructor
    C(size_t size) : size(size), data(new int[size]) {
        std::fill(data, data + size, 0); // Initialize array with zeros
        std::cout << "Constructor called\n";
    }

    // Copy Constructor
    C(const C& other) : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + other.size, data);
        std::cout << "Copy Constructor called\n";
    }

    // Copy Assignment Operator
    C& operator=(const C& other) {
        if (this == &other) {
            return *this; // Self-assignment check
        }

        delete[] data; // Free existing resource

        size = other.size;
        data = new int[other.size];
        std::copy(other.data, other.data + other.size, data);

        std::cout << "Copy Assignment Operator called\n";
        return *this;
    }

    // Move Constructor
    C(C&& other) noexcept : size(other.size), data(other.data) {
        other.size = 0;
        other.data = nullptr;
        std::cout << "Move Constructor called\n";
    }

    // Move Assignment Operator
    C& operator=(C&& other) noexcept {
        if (this == &other) {
            return *this; // Self-assignment check
        }
    }
}
```

```

        delete[] data; // Free existing resource

        size = other.size;
        data = other.data;

        other.size = 0;
        other.data = nullptr;

        std::cout << "Move Assignment Operator called\n";
        return *this;
    }

    // Destructor
    ~C() {
        delete[] data;
        std::cout << "Destructor called\n";
    }

    // Function to print the array
    void print() const {
        for (size_t i = 0; i < size; ++i) {
            std::cout << data[i] << " ";
        }
        std::cout << std::endl;
    }
};

// Main function to demonstrate the class functionality
int main() {
    // Constructor
    C obj1(10);
    obj1.print();

    // Copy Constructor
    C obj2 = obj1;
    obj2.print();

    // Copy Assignment Operator
    C obj3(5);
    obj3 = obj1;
    obj3.print();

    // Move Constructor
    C obj4 = std::move(obj1);
    obj4.print();

    // Move Assignment Operator
    C obj5(5);
    obj5 = std::move(obj2);
    obj5.print();

    return 0;
}

```